E01 | 生成文字为什么还不是工单事实?

本章从摘要入手：**源记录 → 选择输入 → Provider 生成 → 应用检查 → 待核对草稿**。每一步都应保留来源，生成一句“已修复”不能把原工单的 open 改成已关闭。

需要模型增加哪一种能力?

当前问题	关系或短用法	记到什么程度
显示工单状态、消息数，要调用模型吗?	状态从业务记录取，数量用 `len(...)` 算，固定表达用模板；自然语言长短不一、需要整理时，模型才增加新的处理能力。	**确定性查询和计算优先用明确规则。** 不因有 AI 就重写事实获取。
“用户报告离线”与“已检测确认故障”差在哪?	源工单证明用户提交了什么，不单独证明设备此刻状态或故障原因；摘要不能把报告提高成诊断证据。	能看出证据强度被悄悄提高；措辞流畅不补事实。
Prompt 写了“别泄露备注”，可以发送整张工单吗?	应用先白名单选择编号、型号、消息，再构造请求；无关私人备注在调用前排除。	**输入筛选在发送之前。** 指令无法收回已发送字段。
一份结果怎样同时放事实与草稿?	`application_facts` 来自应用读取或计算，`draft.summary` 来自生成；后者不能覆盖业务字段。	按所有者保存；界面应能分清原记录与待核对文字。

回看：「当前已经知道什么，想让模型多做什么？」「一次生成应该从哪里取得输入？」。

程序接受了一段文字，究竟接受到哪一步?

当前问题	关系或短用法	记到什么程度
`accepted_draft` 是否代表摘要已证实?	本例只查 JSON 与非空字符串结构，仍标 `unverified_candidate`；“已修复”也可能过这一关，但源工单仍 open。	**接受草稿不等于确认事实或完成业务写入。**
字段错与超时分别留下什么?	`summary=42` 已取得生成结果，可留 generation ID，草稿为空；timeout 未取得完整结果，不拿旧 generation ID 冒充本次成功。	区分内容拒绝和未取得结果；失败要改变公开状态。
保留旧草稿能让本次刷新看起来成功吗?	若显示旧草稿，须带旧结果身份与时间，同时显示本次失败。	旧值可以保留，但不能失去归属。
request ID 与 generation ID 为什么分开?	前者关联应用处理，后者关联生成结果；一次处理可能包含重试或多次生成。	熟悉两种身份；本地示例字段名不等于供应商统一返回格式。
Fake 两种措辞证明模型效果提高了吗?	两句都是脚本预置；实际运行可证明应用保留事实、接收合格结构、拒绝坏结构与超时。	**本地边界验证与真实语言效果分开。** Fake 不测答案概率、质量或真实费用。

回看: 「用一个本地替身看见完整结果」「为什么已经收到了回应，仍可能没有草稿?」「现在的运行究竟证明了多少?」。

生成机制与当前输入怎样连接?

当前问题	关系或短用法	记到什么程度
发送工单，是把它训练进模型了吗?	训练调整参数；本次推理把选中的资料放进 context，根据可见序列逐步生成 token。调用本身不表示写入长期知识。	分清训练、推理与当前上下文；生成不会自动读取本地数据库或检测设备。
字符数就是 token 数吗? 屏幕历史都可见吗?	tokenizer 把文字编码成片段；token 不固定等于汉字或单词。模型只见本次实际输入，窗口有限。	理解输入选择的边界；窗口、分词器、输入输出计数查当前官方文档，不背过时数字。
降低随机性后答案相同，就能当事实查询吗?	相同措辞只说明这些运行一致，不说明新增了来源证据。	不用稳定输出替代事实读取；需要来源和用途校验时接 E02。

回看: 「流畅的文字是怎样产生的?」。真实接入再查 Provider 的请求、回应、错误、数据处理、成本与上下文合同；密钥不进示例、日志或提交记录。